Reproducibility: producer provenance and recovery checks

Untagged PDF derivative | HTML reader

Producer order, stale invalidation, and publication authority

The provenance map in fig. 23 records the source-owned producer contract rather than inferring it from the release manifest.

Producer order, stale... (part 2)

Panel A places the non-sequential fan-in dependencies in a labeled matrix, then orders the temporal, gate, receipt, and authorization dependencies in a crossing-free process path. Source, configuration, lockfile,

Producer order, stale... (part 3)

figure-accessibility registries, and the exact clean Template commit remain explicit inputs.

Producer order, stale... (part 4)

Panel B's fourteen coded dashed cards make reverse invalidation explicit without crossing the forward process path. Each arrow starts at one or more immediate stale targets on the right and points back to its changed owner on the left.

Producer order, stale... (part 5)

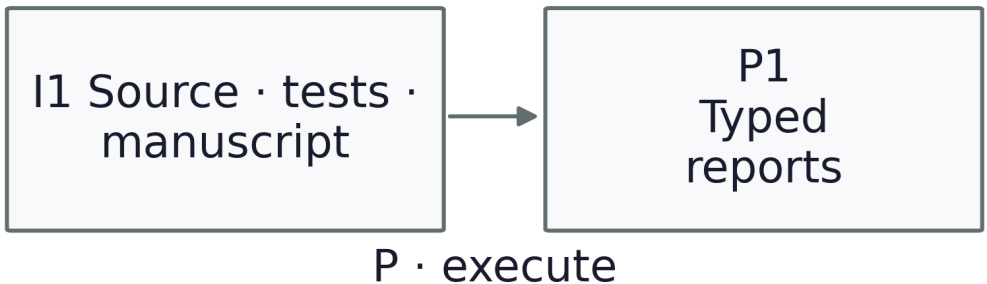
A changed owner makes the named immediate targets and their downstream reports, figures, hydrated text, rendered surfaces, provenance, and release receipts stale; previously green bytes are not carried forward.

Producer order, stale... (part 6)

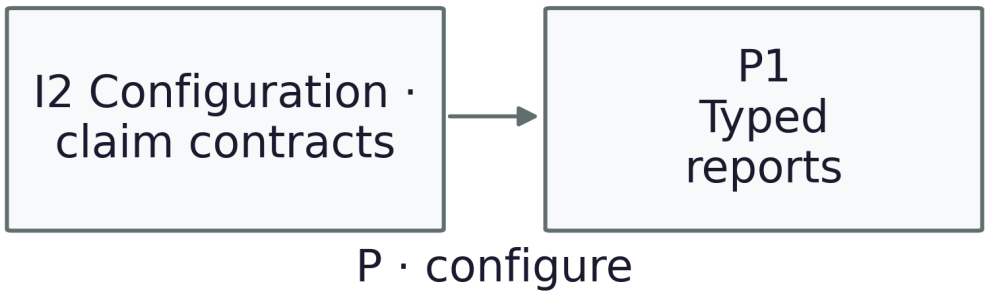
GitHub and Zenodo are terminal authorization states, not automatic products of a green build. The figure itself is generated from the source-owned pipeline contract and never reads the downstream release manifest that later inventories it, avoiding a circular dependency.

Untagged PDF derivative | HTML reader

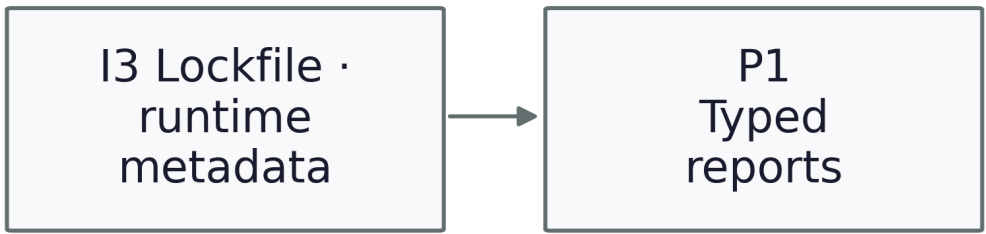
Producer order, stale... (part 7)



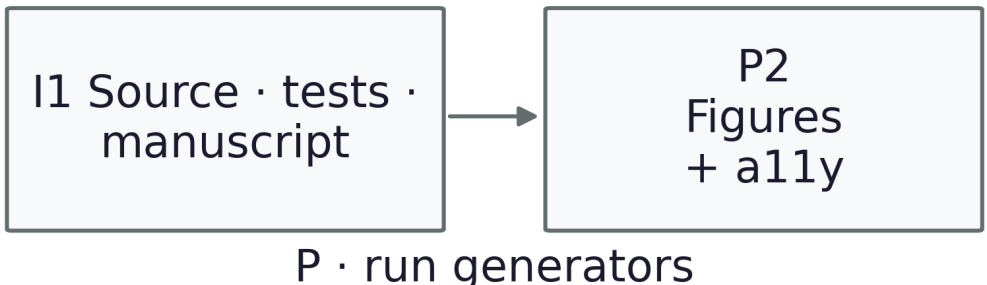
Producer order, stale... (part 8)



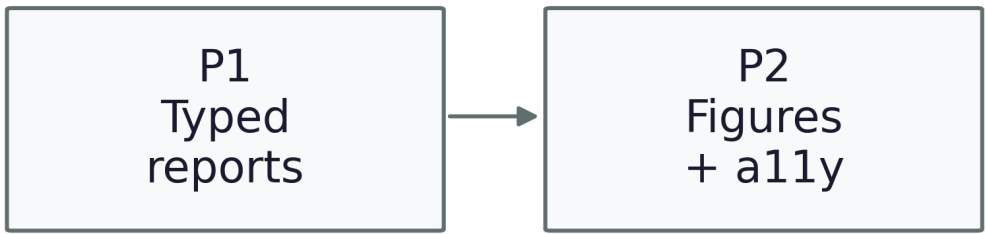
Producer order, stale... (part 9)



Producer order, stale... (part 10)

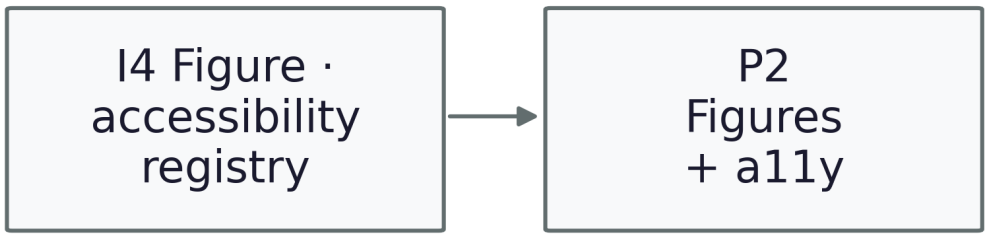


Producer order, stale... (part 11)

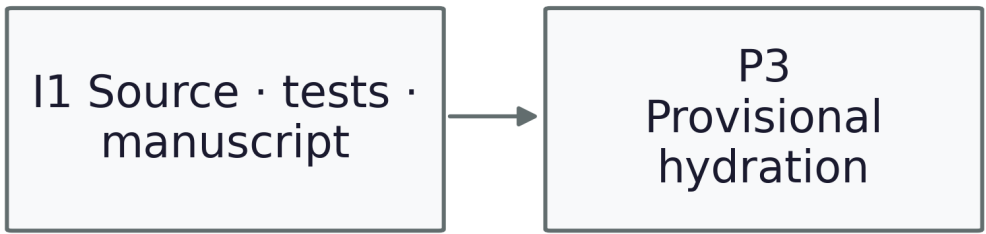


Untagged PDF derivative | HTML reader

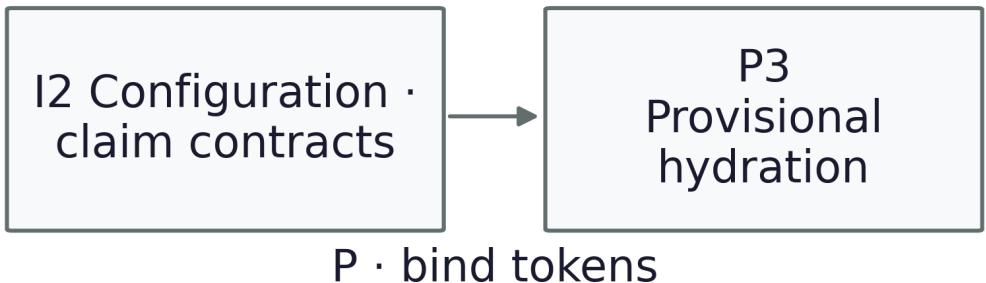
Producer order, stale... (part 12)



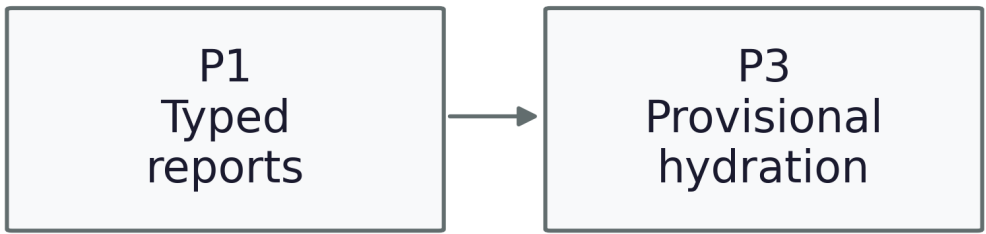
Producer order, stale... (part 13)



Producer order, stale... (part 14)



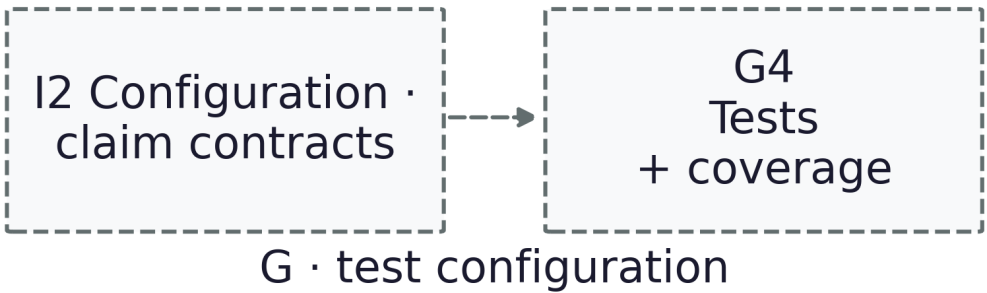
Producer order, stale... (part 15)



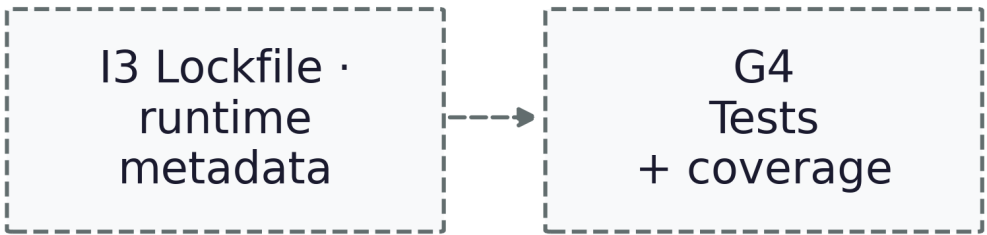
Producer order, stale... (part 16)



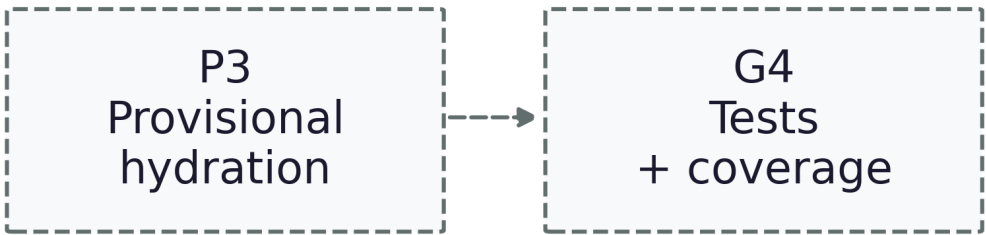
Producer order, stale... (part 17)



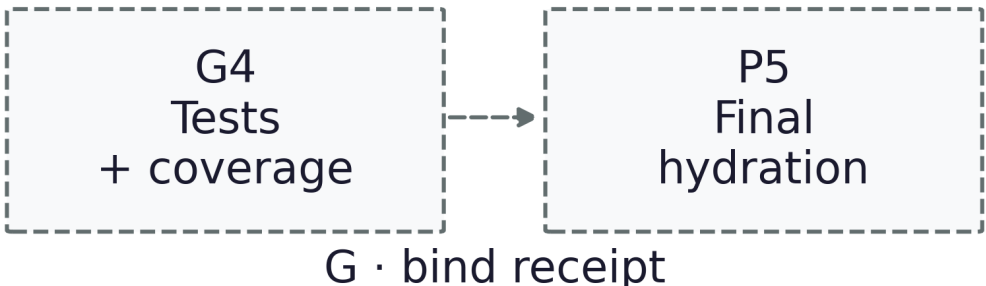
Producer order, stale... (part 18)



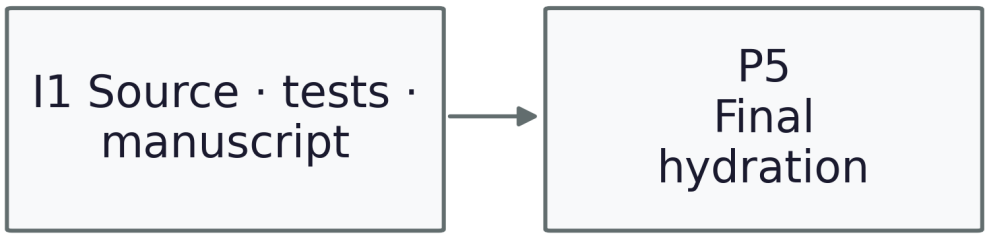
Producer order, stale... (part 19)



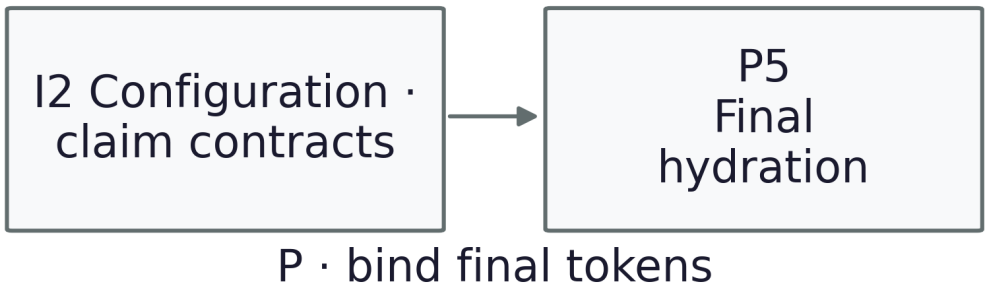
Producer order, stale... (part 20)



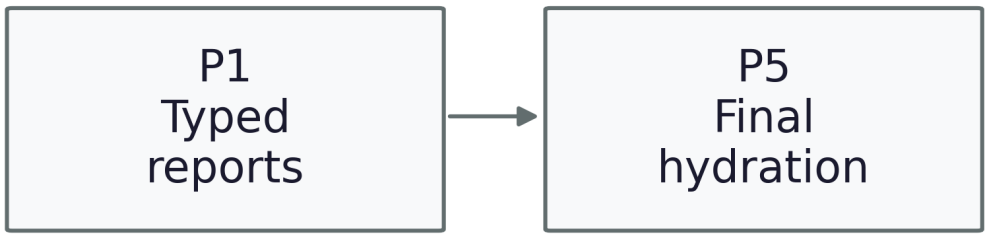
Producer order, stale... (part 21)



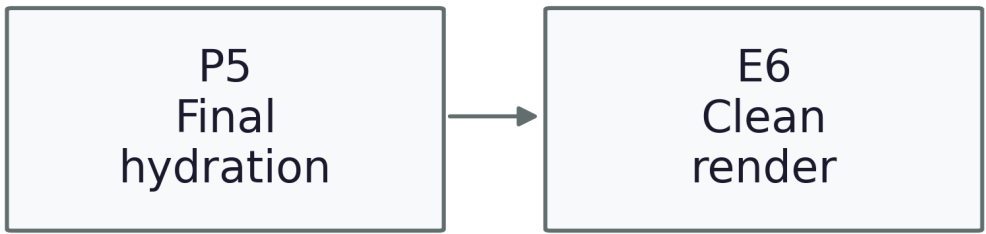
Producer order, stale... (part 22)



Producer order, stale... (part 23)

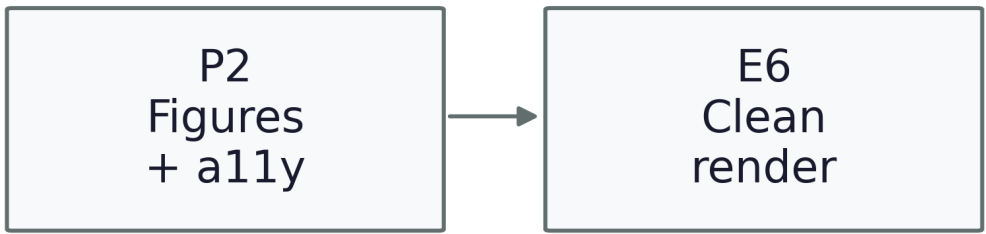


Producer order, stale... (part 24)

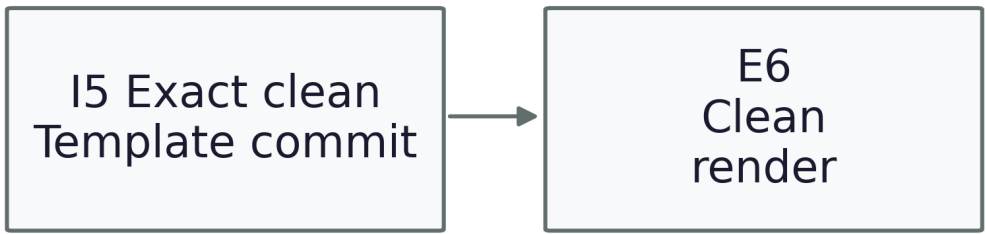


P · render manuscript

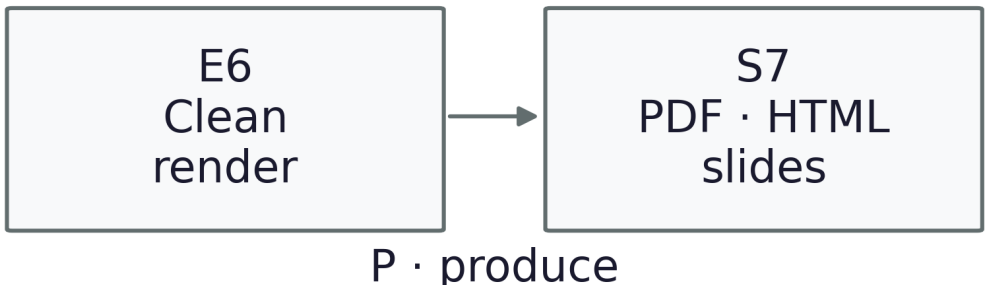
Producer order, stale... (part 25)



Producer order, stale... (part 26)

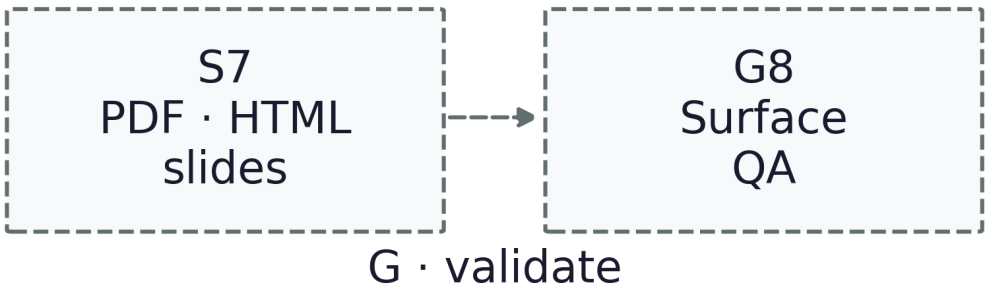


Producer order, stale... (part 27)

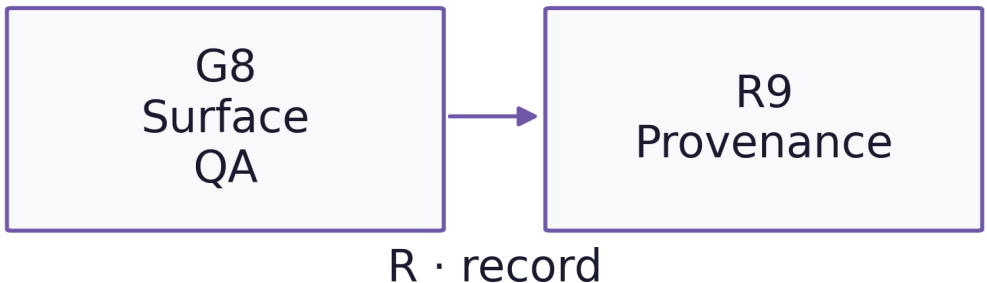


Untagged PDF derivative | HTML reader

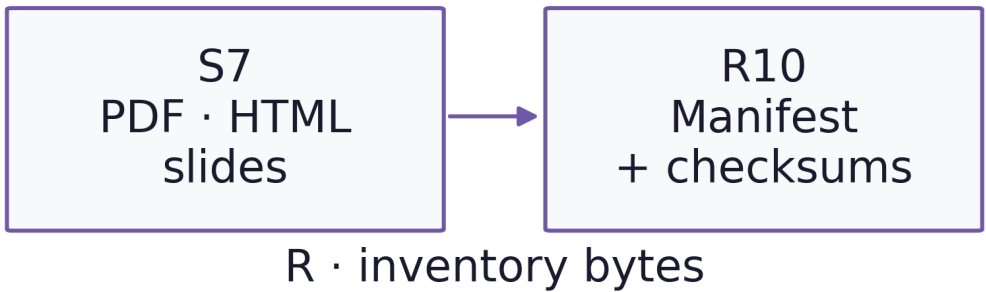
Producer order, stale... (part 28)



Producer order, stale... (part 29)

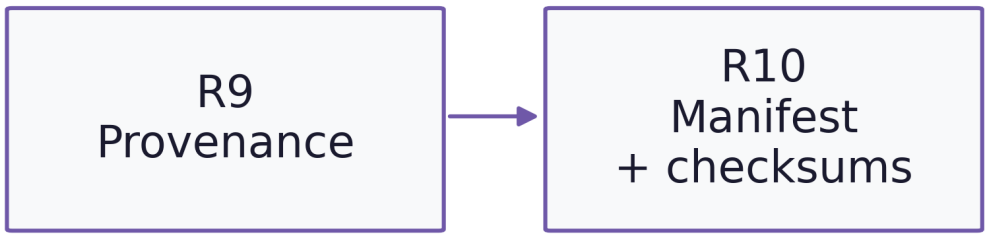


Producer order, stale… (part 30)

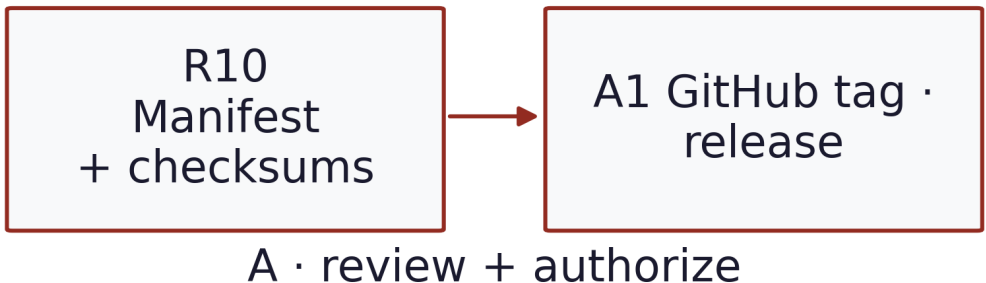


Untagged PDF derivative | HTML reader

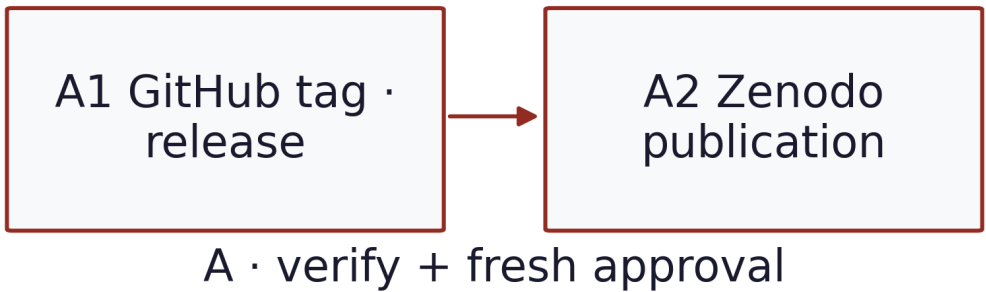
Producer order, stale... (part 31)



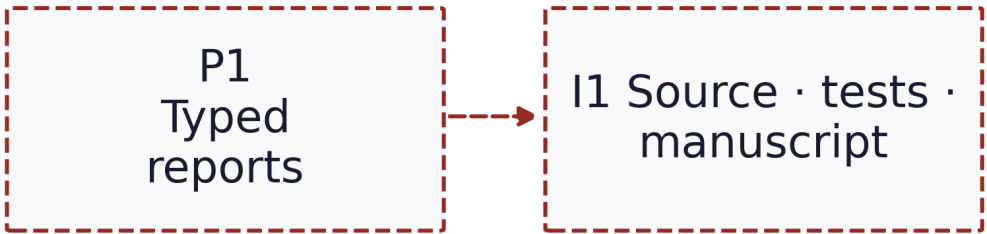
Producer order, stale… (part 32)



Producer order, stale… (part 33)



Producer order, stale... (part 34)



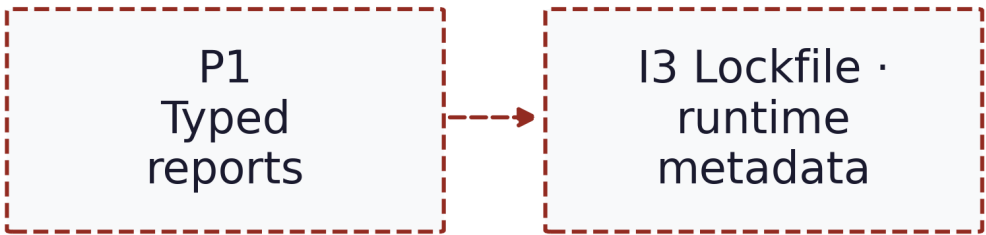
Stale dependency after change

Producer order, stale... (part 35)



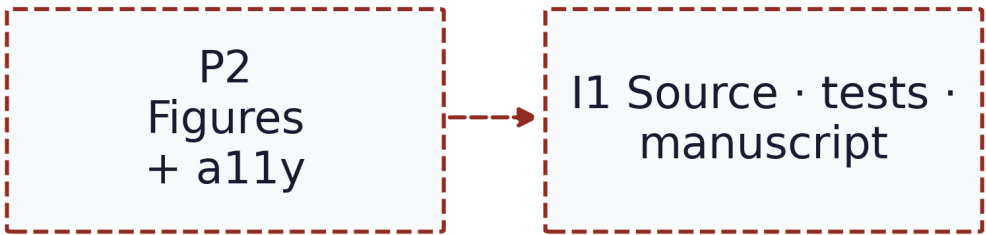
Stale dependency after change

Producer order, stale... (part 36)

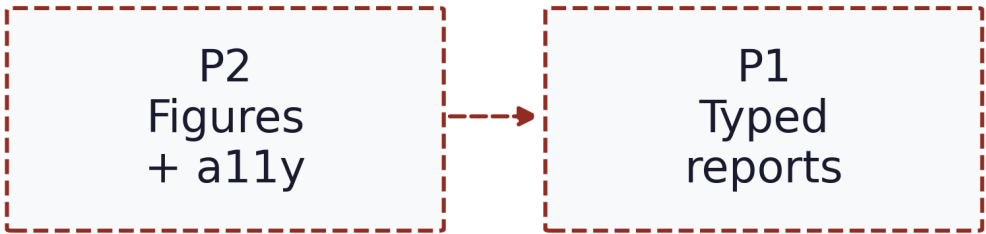


Stale dependency after change

Producer order, stale... (part 37)

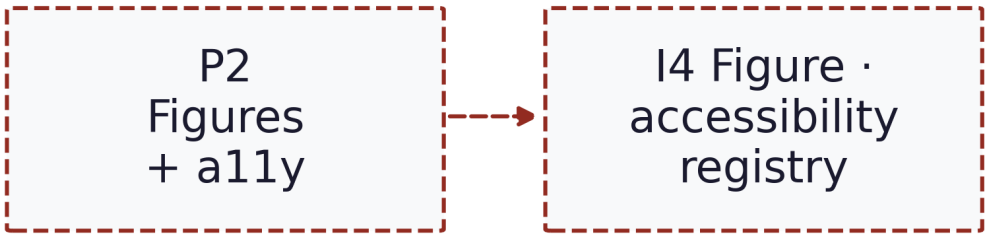


Producer order, stale… (part 38)

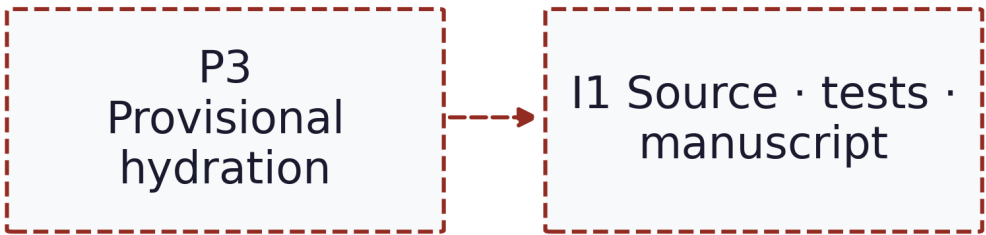


Stale dependency after change

Producer order, stale... (part 39)

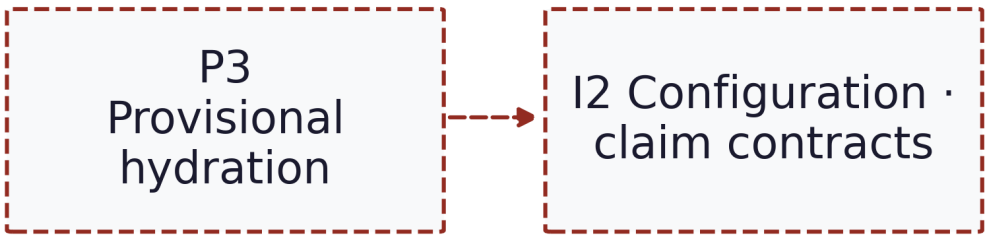


Producer order, stale… (part 40)



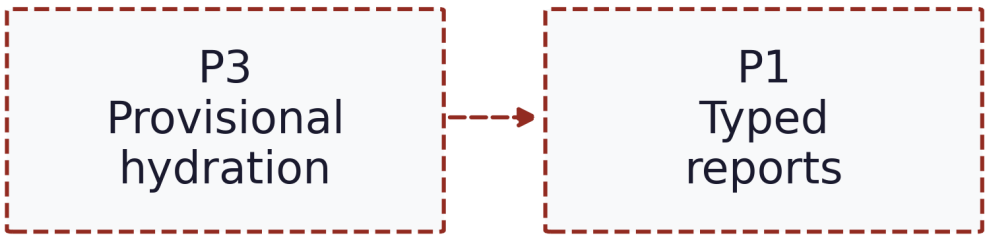
Stale dependency after change

Producer order, stale... (part 41)

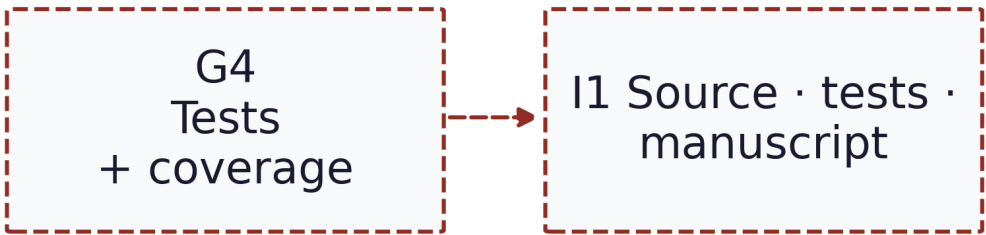


Stale dependency after change

Producer order, stale... (part 42)



Producer order, stale... (part 43)



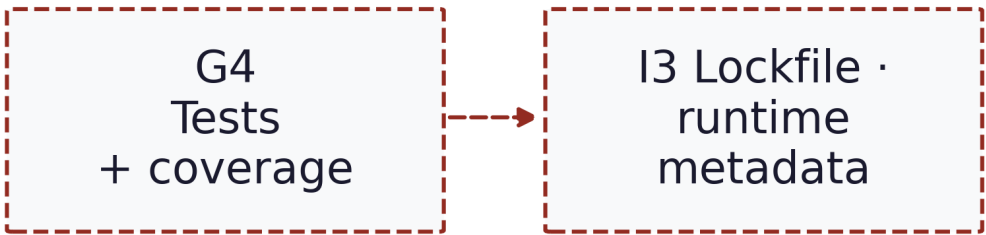
Stale dependency after change

Producer order, stale... (part 44)



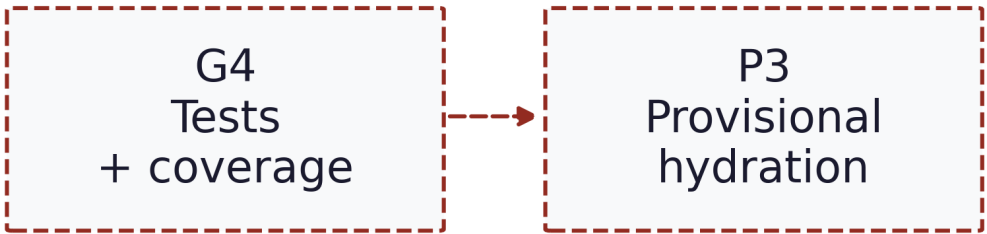
Stale dependency after change

Producer order, stale... (part 45)



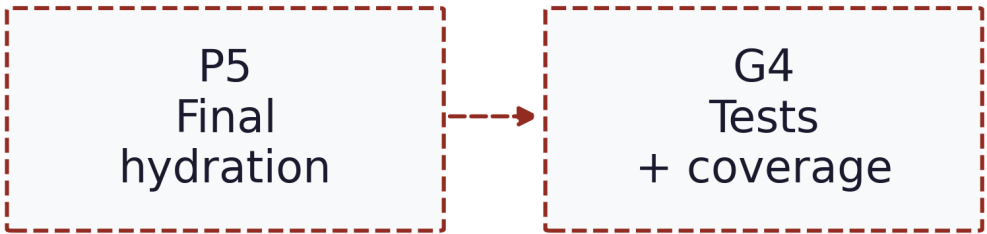
Stale dependency after change

Producer order, stale... (part 46)

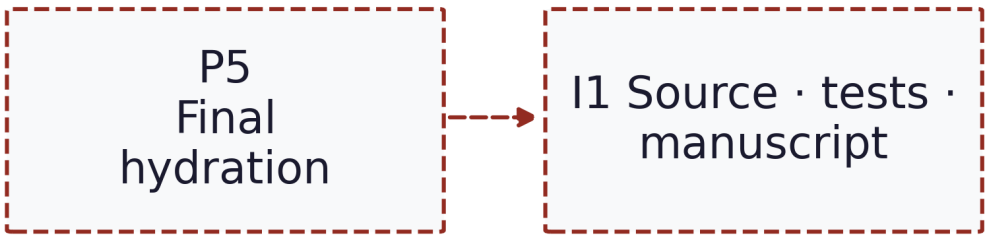


Stale dependency after change

Producer order, stale... (part 47)

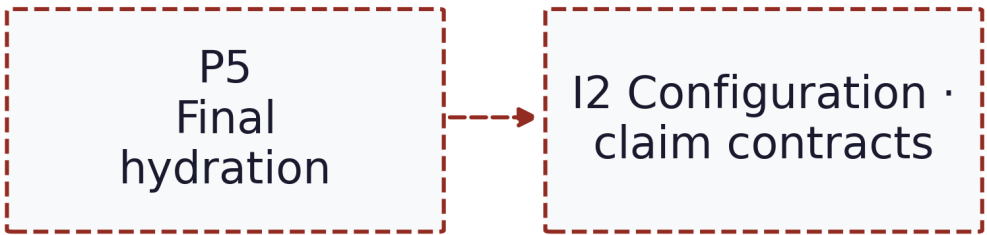


Producer order, stale... (part 48)

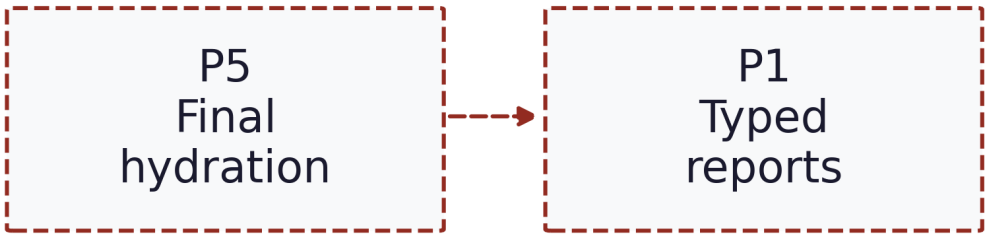


Stale dependency after change

Producer order, stale… (part 49)

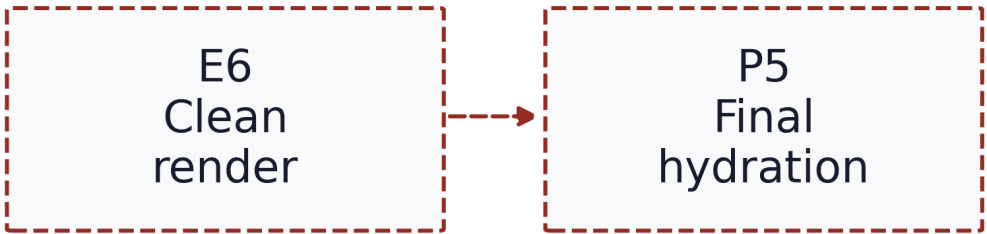


Producer order, stale... (part 50)

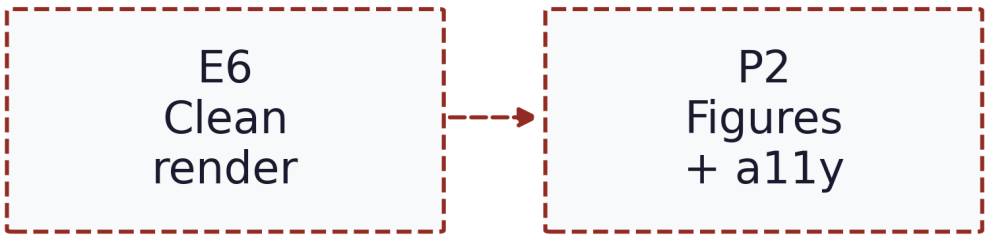


Stale dependency after change

Producer order, stale... (part 51)

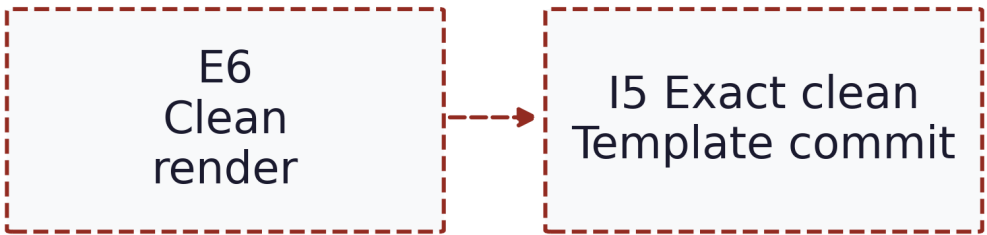


Producer order, stale... (part 52)

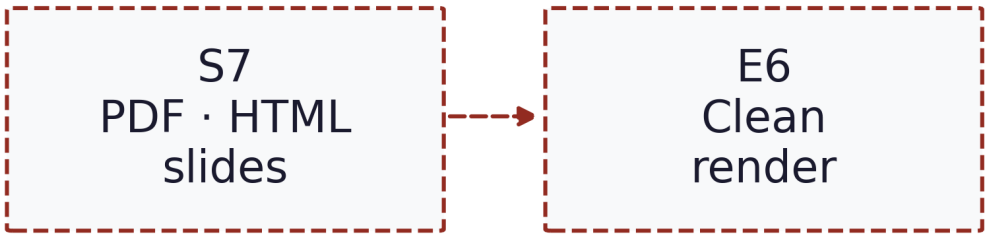


Stale dependency after change

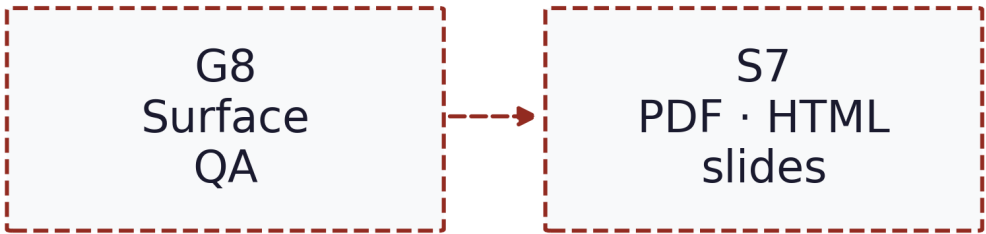
Producer order, stale... (part 53)



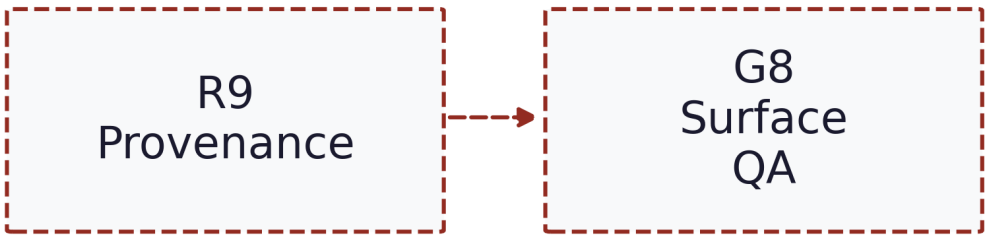
Producer order, stale... (part 54)



Producer order, stale... (part 55)

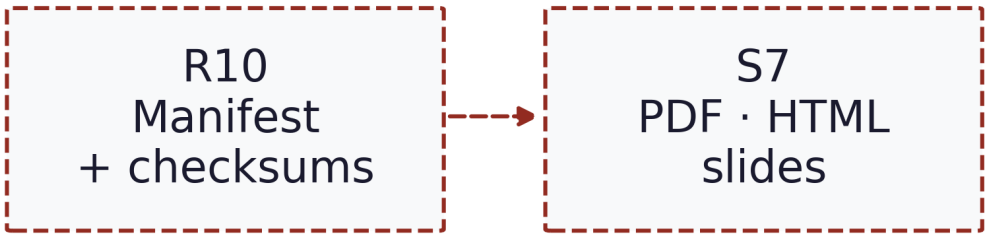


Producer order, stale... (part 56)

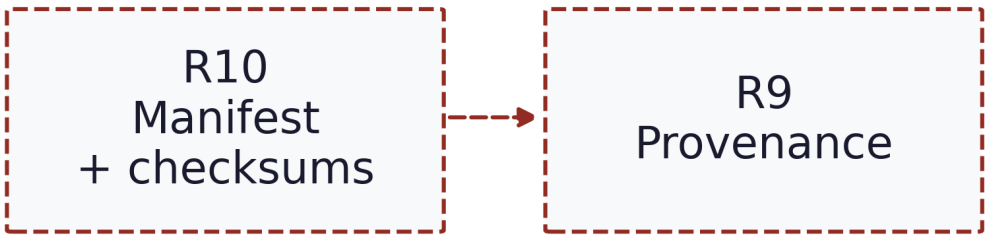


Stale dependency after change

Producer order, stale… (part 57)



Producer order, stale... (part 58)



Stale dependency after change

Producer order, stale… (part 59)

green engineering and rendering gates are necessary inputs to review; they do not automatically authorize a merge, tag, GitHub release, or Zenodo publication

Untagged PDF derivative | HTML reader

Producer order, stale... (part 60)

this figure is generated from this source-owned contract and never reads the downstream release manifest that later inventories it

Producer order, stale... (part 61)

a green build is not scientific validation

Producer order, stale... (part 62)

tagged PDF structure is not PDF/UA conformance

Untagged PDF derivative | HTML reader

Producer order, stale… (part 63)

HTML accessibility enhancements are not a WCAG conformance claim

Untagged PDF derivative | HTML reader

Producer order, stale… (part 64)

a GitHub release or DOI does not enlarge the scientific claim surface

Recovery-limit certificate for the client and project-pool corners

The recovery identities are reproducibility checks: the client machinery must return to standard Bayes at its KL/NLL loss limits,

Recovery-limit certificate... (part 2)

and the server heuristic must return to the project's log-linear pool at zero robustness (eq. 10, eq. 11).

Recovery-limit certificate... (part 3)

Under the explicit shared-support, posterior-log-potential, and fixed-weight assumptions of sec. 8.8, that pool specializes Eq. 7's message-combination term; it does not reproduce the complete source protocol. These deviations are computed on every build.

Recovery-limit certificate... (part 4)

Server recovery:
`robust_aggregate(robustness=0)` versus `log_linear_pool` (eq. 9) has maximum difference 0.

Recovery-limit certificate... (part 5)

Posterior recovery:
`generalized_posterior(KLD, NLL)` versus closed-form Bayes has maximum difference 5.55e-17.

Divergence recovery: Rényi divergence versus KL as $\alpha \to 1$ has maximum difference 0.

Recovery-limit certificate... (part 6)

Loss recoveries: β-loss versus NLL as $\beta \to 0$ has maximum difference 0, while rcce versus NLL as $q_{\text{loss}} \to 0$ has maximum difference 0.

Recovery-limit certificate... (part 7)

Any drift in these limits beyond machine precision would mean the robust generalization no longer contains its standard-Bayes client limit and project-local log-linear-pool server limit (sec. 10.1) — and would fail the core test suite before this certificate could render.

Recovery-limit certificate... (part 8)

The certificate covers the recovery identity and the client-side result under the cited source theorem's matching assumptions only; the server-side `robust_aggregate` heuristic is certified here for its recovery limit alone, not for any bounded-influence property (sec. 11.13).

Recovery-limit certificate... (part 9)

All code is authored by Daniel Ari Friedman and licensed under the MIT license. This is project version 1.1.0.